Universidad San Carlos de Guatemala

Base de Datos 2

Ing. Daniel Gonzales



MANUAL TÉCNICO

Mario Raúl Yancor Ocaña

201930761

Quetzaltenango, 30 de octubre de 2026

Introducción

Este manual describe la estructura técnica, instalación, configuración, ejecución y validación del sistema desarrollado para analizar resultados de pruebas específicas de la Universidad de San Carlos de Guatemala. La solución integra una base documental en MongoDB y una base de grafos en Neo4j, con una API desarrollada en Node.js y una interfaz web en React.

El documento está dirigido a la persona que necesite instalar el proyecto desde cero, comprender su funcionamiento interno, ejecutar las consultas o darle mantenimiento.

Objetivos y alcance

- Documentar la arquitectura y las tecnologías utilizadas.
- Explicar el almacenamiento de datos en MongoDB y Neo4j.
- Describir el proceso de carga y transformación de archivos CSV.
- Presentar los endpoints y las consultas implementadas.
- Proporcionar una guía reproducible de instalación, ejecución y pruebas.

El sistema permite cargar archivos CSV, generar indicadores estadísticos, ejecutar veinte consultas en MongoDB, ejecutar cinco consultas en Neo4j y explorar las relaciones de un aspirante.

Arquitectura del sistema

La aplicación utiliza una arquitectura cliente servidor. El frontend consume la API mediante solicitudes HTTP. El backend valida los archivos y distribuye la información entre MongoDB y Neo4j. Docker Compose administra los dos motores de base de datos.

Capa	Componente	Responsabilidad
Presentación	React y Vite	Muestra dashboard, gráficas, formularios y resultados.
API	Node.js y Express	Recibe solicitudes, valida datos y ejecuta consultas.
ETL	Servicio de carga CSV	Lee, limpia, transforma y almacena registros.
Documental	MongoDB	Conserva evaluaciones y soporta agregaciones estadísticas.
Grafos	Neo4j	Representa aspirantes, evaluaciones y relaciones.
Infraestructura	Docker Compose	Levanta y comunica los servicios de datos.

Flujo general

1. El usuario selecciona uno o varios archivos CSV desde React.
2. La API recibe cada archivo mediante una petición multipart.
3. El servicio ETL valida y transforma los campos.
4. Los registros se almacenan en MongoDB y se crean nodos y relaciones en Neo4j.
5. El frontend consulta la API y presenta los resultados.

Tecnologías utilizadas

Tecnología	Uso
Node.js 22	Entorno de ejecución del backend.
Express	Definición de la API REST.
Multer	Recepción de archivos CSV en memoria.
MongoDB	Persistencia documental y agregaciones.
Mongoose	Modelado y acceso a MongoDB.
Neo4j	Persistencia y consulta del grafo.
neo4j-driver	Conexión desde Node.js hacia Neo4j.
React	Construcción de la interfaz.
Vite	Servidor de desarrollo y compilación.
Axios	Consumo de la API.
Recharts	Visualización de gráficas.
Docker Compose	Ejecución de MongoDB y Neo4j.
Git y GitHub	Control de versiones mediante ramas.

Puertos utilizados

Servicio	Puerto	Dirección
Frontend	5173	http://localhost:5173
Backend	3000	http://localhost:3000/api
MongoDB	27017	mongodb://localhost:27017
Neo4j HTTP	7474	http://localhost:7474
Neo4j Bolt	7687	bolt://localhost:7687

Estructura del proyecto

```
BD2_Proyecto_Final/
  backend/
      scripts/
      src/
      config/
      controllers/
      models/
      routes/
      services/
      server.js
  data/raw/
  database/mongodb/consultas/
  database/neo4j/consultas/
  docs/
  frontend/
      src/components/
      src/App.jsx
      src/App.css
  docker-compose.yml
  README.md
```

El backend separa las conexiones, controladores, rutas, modelos y servicios. El frontend contiene la vista principal y componentes especializados para el grafo y el explorador de consultas. La carpeta database conserva las consultas documentadas y docs contiene los manuales.

Estrategia de ramas

El trabajo se realiza en ramas feature. Cada funcionalidad terminada se confirma y se sube a GitHub. Después se integra en develop mediante un merge sin avance rápido. La rama main se actualiza al finalizar el proyecto.

```
feature/* -> develop -> main
```

Modelo de datos

MongoDB

La base de datos utilizada por el proyecto es bd2_proyecto_final. Mongoose creó la colección evaluacions, donde cada documento representa una evaluación realizada por un aspirante.

Campo	Descripción
fecha_asignacion	Fecha asignada para la evaluación.
sexo	Sexo registrado.
anio_nacimiento	Año de nacimiento.
edad	Edad calculada respecto al año de ingreso.
materia	Matemática o Computación.

numero_de_fecha_de_evaluacion	Número de intento.
anio_de_ingreso	Año de ingreso.
aprobacion	Resultado textual.
aprobado	Resultado booleano.
carrera_objetivo	Carrera seleccionada.
departamento_institucion_educativa	Departamento de procedencia.
municipio_institucion_educativa	Municipio de procedencia.
tipo_institucion_educativa	Tipo de institución.
correlativo_aspirante	Identificador del aspirante.
archivo_origen	Nombre del CSV procesado.

Neo4j

Neo4j representa la información mediante nodos y relaciones. Este modelo permite recorrer fácilmente la información de un aspirante y obtener sus evaluaciones, carrera y procedencia.

Etiqueta	Propiedades principales
Aspirante	correlativo, sexo
Evaluacion	intento, aprobado, aprobación, fecha y año
Materia	nombre
Carrera	nombre
Municipio	nombre
Departamento	nombre
TipoInstitucion	nombre
Relación	**Origen y destino**
REALIZO	Aspirante hacia Evaluacion
EVALUA	Evaluacion hacia Materia
ASPIRA_A	Aspirante hacia Carrera
PROVIENE_DE	Aspirante hacia Municipio
ESTUDIO_EN_TIPO	Aspirante hacia TipoInstitucion
PERTENECE_A	Municipio hacia Departamento

Después de la carga inicial se obtuvieron 6,148 aspirantes, 20,277 evaluaciones, 135 municipios, 24 departamentos, 5 tipos de institución, 2 materias y 2 carreras.

Proceso ETL

El proceso ETL recibe el archivo, valida su extensión, lee las filas, normaliza los textos, convierte números y fechas, calcula la edad y transforma el resultado de aprobación a booleano. Después inserta cada evaluación en MongoDB y actualiza el grafo en Neo4j.

6. Extracción: lectura del CSV recibido mediante Multer.
7. Transformación: limpieza, normalización, cálculo de edad y validación de campos.
8. Carga documental: inserción en la colección evaluacions.
9. Carga de grafos: creación o reutilización de nodos y relaciones.
10. Respuesta: resumen con leídos, procesados, rechazados, errores y tiempo.

Prevención de duplicados

Antes de procesar un archivo se consulta el campo archivo_origen. Si ya existe un registro con el mismo nombre, la API devuelve el estado HTTP 409 y no vuelve a insertar la información. Esta validación evita que los indicadores se dupliquen.

Resultado de la carga inicial

Métrica	Resultado
Archivo	pruebas_especificas_2015.csv
Leídos	20,277
Procesados correctamente	20,277
Rechazados	0
Estado	FINALIZADO

API y consultas

La API se publica bajo la ruta http://localhost:3000/api. Las rutas se agrupan por carga de archivos, consultas MongoDB, consultas Neo4j, administración y verificación del servicio.

Rutas generales

Método	Ruta	Función
GET	/api/health	Verifica el estado del backend.
POST	/api/uploads/csv	Procesa un archivo CSV.
DELETE	/api/admin/datos	Elimina los datos de MongoDB y Neo4j.

Consultas MongoDB

N.	Método	Ruta
1	GET	/api/mongodb/aspirantes-tipo-institucion
2	GET	/api/mongodb/aprobados-materia
3	GET	/api/mongodb/aprobados-carrera-anio

4	GET	/api/mongodb/porcentaje-aprobacion-materia
5	GET	/api/mongodb/promedio-edad-carrera
6	POST	/api/mongodb/resumen-carrera
7	GET	/api/mongodb/edad-aprobados-carrera-tipo
8	GET	/api/mongodb/aprobados-municipio-carrera
9	GET	/api/mongodb/evaluaciones-mes-materia-publicas
10	GET	/api/mongodb/top-carreras-16-18
11	GET	/api/mongodb/distribucion-sexo-tipo
12	GET	/api/mongodb/tasa-aprobacion-edad
13	GET	/api/mongodb/promedio-intentos-materia
14	GET	/api/mongodb/historial-desempeno/:correlativo
15	GET	/api/mongodb/historial-completo/:correlativo
16	GET	/api/mongodb/carreras-reprobados-primer-intento
17	GET	/api/mongodb/top-municipios-aspirantes
18	GET	/api/mongodb/tasa-aprobacion-tipo-institucion
19	GET	/api/mongodb/carreras-demandadas-departamento
20	GET	/api/mongodb/evolucion-aprobacion-anio

Consultas Neo4j

N.	Método	Ruta
1	GET	/api/neo4j/aspirantes-carrera
2	GET	/api/neo4j/aprobados-materia
3	GET	/api/neo4j/aspirantes-municipio
4	GET	/api/neo4j/aspirantes-tipo-institucion
5	GET	/api/neo4j/grafo-aspirante/:correlativo

Ejemplo de consumo

```
curl.exe http://localhost:3000/api/mongodb/evolucion-aprobacion-anio
curl.exe http://localhost:3000/api/neo4j/aprobados-materia
curl.exe http://localhost:3000/api/neo4j/grafo-aspirante/949D241AC10256EDF11339A303C72A50
```

Las respuestas se entregan en formato JSON. El frontend transforma las claves técnicas en nombres legibles y presenta los datos en tablas o gráficas.

Interfaz web

La interfaz centraliza el dashboard, la carga de archivos, el explorador de grafo y el explorador de consultas. Axios comunica React con la API y Recharts dibuja las gráficas.



Figura 1. Encabezado del dashboard.

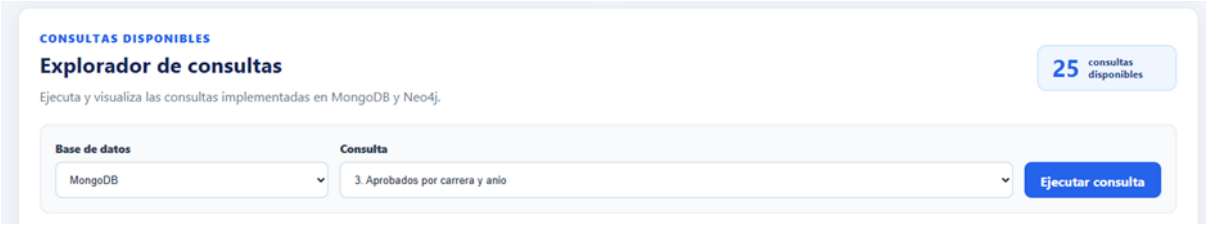


Figura 2. Explorador con 25 consultas disponibles.

El explorador utiliza un controlador de cancelación y un límite de espera. Esto permite cancelar una consulta que tarda demasiado y seleccionar otra sin dejar bloqueada la interfaz.

Guía de instalación y ejecución desde cero

Requisitos

Windows 10 u 11.
Node.js 22 o una versión compatible.
npm 10 o superior.
Git.
Docker Desktop con Docker Compose.
Visual Studio Code u otro editor.

Clonar el repositorio

```
git clone https://github.com/MaYan711/BD2-ProyectoFinal.git
cd BD2-ProyectoFinal
```

Configurar el backend

```
cd backend
npm install
```

Se debe crear backend/.env con las variables utilizadas por el servidor. Los nombres deben coincidir con las variables leídas en los archivos de configuración.

```
PORT=3000
MONGODB_URI=mongodb://localhost:27017/bd2_proyecto_final
NEO4J_URI=bolt://localhost:7687
NEO4J_USER=neo4j
NEO4J_PASSWORD=contraseña_configurada
```

El archivo .env no debe subirse al repositorio. Puede agregarse un archivo .env.example sin datos sensibles.

Levantar las bases de datos

Desde la raíz del proyecto se ejecuta Docker Compose:

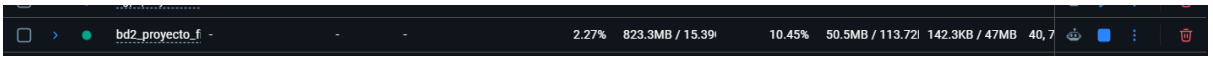
```
docker compose up -d
```

El estado de los contenedores se verifica con:

```
docker compose ps
```

Los contenedores esperados son bd2_mongodb y bd2_neo4j. Si alguno no aparece como activo, se revisan los registros:

```
docker compose logs mongodb
docker compose logs neo4j
```



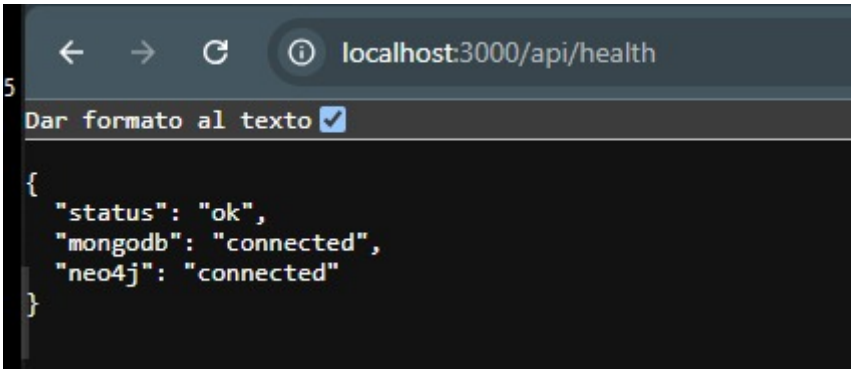
Iniciar el backend

```
cd backend
npm run dev
```



La salida esperada indica que la API se ejecuta en http://localhost:3000. La conexión se comprueba con:

curl.exe http://localhost:3000/api/health



Iniciar el frontend

cd frontend
npm install
npm run dev



Vite publica la aplicación en http://localhost:5173. El backend y el frontend deben permanecer activos en terminales diferentes.

Cargar los datos iniciales

```
curl.exe -X POST http://localhost:3000/api/uploads/csv -F "file=@data/raw/pruebas_especificas_2015.csv"
```

La respuesta correcta informa 20,277 registros leídos, 20,277 procesados y cero rechazados. La carga puede tardar más de un minuto porque actualiza dos bases de datos.

Reinicio completo de datos

Si se necesita repetir la carga desde cero, primero se ejecuta el endpoint administrativo y después se vuelve a enviar el archivo CSV.

```
curl.exe -X DELETE http://localhost:3000/api/admin/datos
curl.exe -X POST http://localhost:3000/api/uploads/csv -F "file=@data/raw/pruebas_especificas_2015.csv"
```

Detener servicios

```
docker compose down
```

Este comando detiene los contenedores. Los volúmenes se conservan mientras no se eliminen expresamente.

Ejecución y pruebas

Validación del frontend

```
cd frontend
 npm run lint
 npm run build
```

El lint debe terminar sin errores y la compilación debe crear la carpeta dist. La advertencia sobre el tamaño de los paquetes no impide la generación del proyecto.

Validación sintáctica del backend

```
cd backend
 node --check src/controllers/uploadController.js
 node --check src/services/etlService.js
```

Prueba automatizada de consultas

El script backend/scripts/probar-consultas.ps1 ejecuta las veinte consultas MongoDB y las cinco consultas Neo4j. Debe ejecutarse con el backend y las bases de datos activos.

```
powershell -ExecutionPolicy Bypass -File .\backend\scripts\probar-consultas.ps1
```

Resultado	Cantidad
Exitosas	25
Fallidas	0
Total	25

Verificación directa de datos

```
docker exec -it bd2_mongodb mongosh bd2_proyecto_final --quiet --eval "db.evaluacions.countDocuments()"
```

El resultado esperado después de la carga limpia es 20277.

```
docker exec -it bd2_neo4j cypher-shell -u neo4j -p CONTRASENA "MATCH (n) RETURN labels(n), count(n);"
```

Esta consulta permite confirmar la cantidad de nodos por etiqueta. La contraseña debe sustituirse por la configurada localmente.

Prueba de duplicados

11. Cargar un archivo CSV de prueba.
12. Intentar cargar nuevamente el mismo archivo.
13. Confirmar que la segunda petición devuelve HTTP 409.
14. Verificar que el total de registros no aumentó.

Esta prueba confirmó que el control por archivo_origen evita la duplicación accidental de evaluaciones.

Análisis de resultados

La carga inicial procesó los 20,277 registros sin rechazos. Esto demuestra que la estructura del archivo fue interpretada correctamente y que el proceso pudo almacenar la misma fuente de información en dos modelos diferentes.

MongoDB fue adecuado para agregaciones estadísticas, porcentajes, promedios y agrupaciones. Neo4j facilitó la navegación desde un aspirante hacia sus evaluaciones, materias, carrera y procedencia. El uso conjunto de ambos motores permitió aplicar persistencia políglota según el tipo de consulta.

Resultado observado	Interpretación
4,749 evaluaciones aprobadas	Representan el 23.42 % de las evaluaciones.
15,528 no aprobadas	Representan el 76.58 %.
Computación con 3,352 aprobados únicos en Neo4j	Presentó más aprobados que Matemática.
Matemática con 1,397 aprobados únicos en Neo4j	Tuvo menor cantidad de aprobados.
Instituciones privadas con 5,133 aspirantes	Concentran la mayor procedencia.
Guatemala zona 1 con 840 aspirantes	Fue el municipio con mayor cantidad.

Las cantidades de MongoDB y Neo4j deben interpretarse según la unidad contada. MongoDB puede contar evaluaciones, mientras algunas consultas Neo4j cuentan aspirantes o relaciones únicas. Por esta razón no toda diferencia significa un error.

Conclusiones

- El proyecto integró correctamente MongoDB y Neo4j dentro de una misma aplicación web.
- El proceso ETL transformó y almacenó 20,277 registros sin rechazos durante la carga inicial.
- MongoDB respondió adecuadamente a las consultas estadísticas y Neo4j permitió representar las relaciones entre las entidades.
- La interfaz facilitó la carga, consulta y visualización de información sin utilizar directamente las consolas de las bases de datos.
- Las 25 consultas fueron validadas exitosamente y no se registraron fallos en la prueba final.
- La prevención de archivos duplicados protegió la consistencia de los indicadores.
- La separación por capas y el uso de Docker permiten reproducir y mantener el proyecto con mayor facilidad.